

Svelte **Review Cards**



Eight concepts an interviewer can probe and a beginner must own.
Read the question, cover the answer, answer aloud, then check the
code and the figure.

1 — Reactivity: \$state

STATE MANAGEMENT

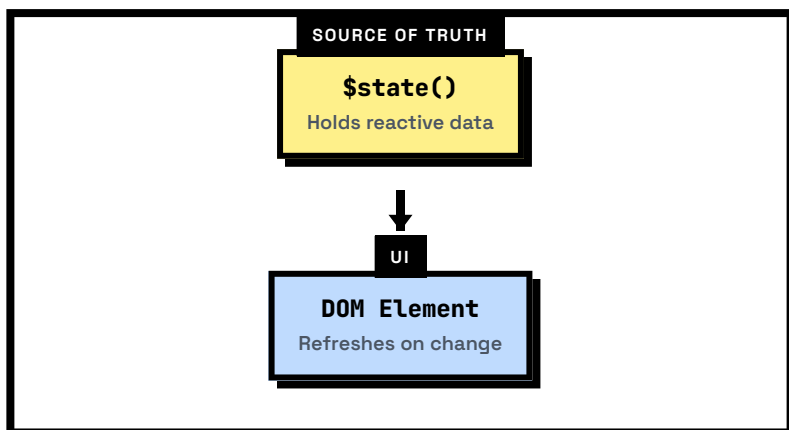
Q. How do you declare a variable that automatically updates the DOM when its value changes in Svelte 5?

ANSWER. Normally, JavaScript variables are completely dead—changing a `let` variable doesn't update your UI. You use the `$state` rune to wrap your initial value, turning it into a live variable that Svelte tracks. Any time that value changes, Svelte instantly updates the DOM for you.

```
<script>
  let count = $state(0);
  let user = $state({ name: 'Nikos' });

  function update() {
    count++;
    user.name = 'Hugo'; // Deeply reactive
  }
</script>
```

App.svelte



- ① **Summary.** The `$state` rune tells Svelte to watch a variable closely. This matters because it automates your UI. Any time you change that variable, Svelte instantly updates the screen for you. You never have to manually touch the DOM again.

2 — Derived Values: \$derived

COMPUTED VALUES

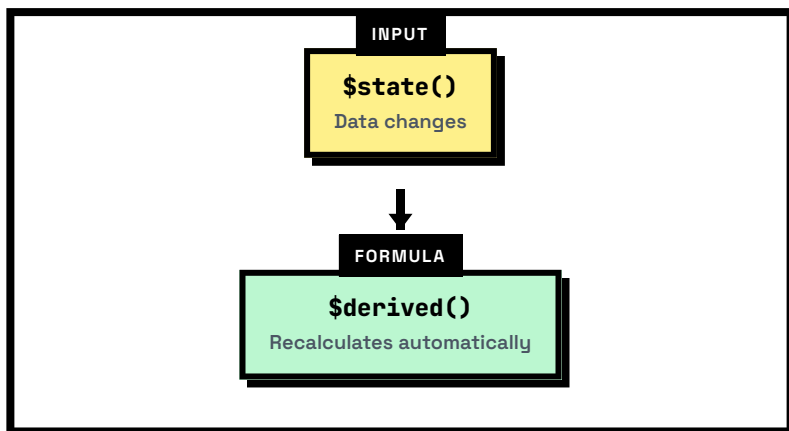
Q. How do you compute a value based on other reactive state so that it updates automatically without manual recalculation?

ANSWER. When building a shopping cart, you need the **totalPrice** to update automatically whenever items are added or removed. Instead of manually recalculating the total every time, you use the **\$derived** rune to create an auto-updating formula. Svelte caches the result and only re-runs the math when the underlying items actually change.

App.svelte

```
<script>
  let count = $state(1);
  let doubled = $derived(count * 2);

  // Derived from objects
  let user = $state({ first: 'Ada', last: 'Lovelace'
});
  let full = $derived(`${user.first} ${user.last}`);
</script>
```



i Summary. Think of **\$derived** as an auto-updating formula. This is powerful because it calculates a value once and caches it. It only re-runs when the inputs actually change. This keeps your app

blazing fast without manual recalculation logic.

3 — Side Effects: \$effect

LIFECYCLE & TRACKING

Q. How do you synchronize an external system (like a canvas or local storage) with your internal reactive state?

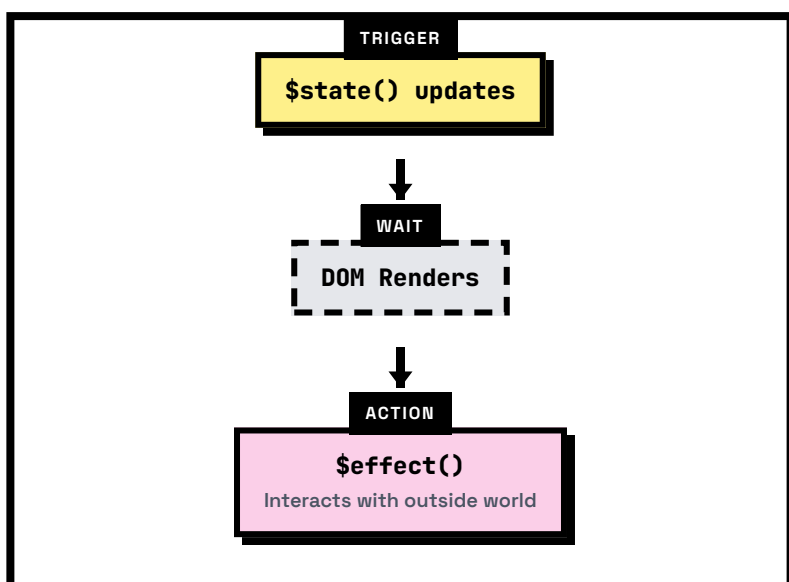
ANSWER. If you need to draw on an HTML `<canvas>` or save a user's theme preference to `localStorage` every time they click a button, plain Svelte variables can't reach those outside APIs automatically. You use the `$effect` rune to run a side effect that automatically tracks its dependencies and runs exactly when needed.

App.svelte

```
<script>
  let canvas = $state();

  $effect(() => {
    if (!canvas) return;
    const ctx = canvas.getContext('2d');

    // Runs automatically after DOM updates
    // whenever 'canvas' changes.
  });
</script>
```



① **Summary.** **\$effect** is your bridge to the outside world. This is essential because it automatically tracks the variables you use inside it. This guarantees that things like analytics, local storage, or canvas drawings always stay perfectly synced with your state.

4 — Component Inputs: \$props

COMPONENT COMMUNICATION

Q. How do you accept inputs from a parent component while maintaining type safety and reactivity?

ANSWER. When building a reusable `<Button>` component, you need a way to pass in text and colors from the parent. You use the `$props` rune to securely accept that data. Destructuring it immediately gives you clean, type-safe variables that stay reactive if the parent changes them.

Parent.svelte

```
<script>
  import Child from './Child.svelte';
</script>

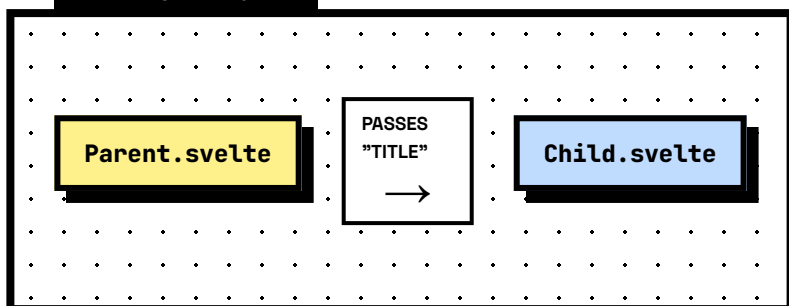
<Child title="Hello Svelte!" />
```

Child.svelte

```
<script>
  let { title } = $props();
</script>

<h1>{title}</h1>
```

DATA FLOW DIAGRAM



i Summary. You now receive data from parent components using the `$props` rune. This is great because destructuring it immediately gives you clean, type-safe inputs. It also totally separates your component's internal state from the outside world.

5 — Avoiding Prop Drilling

ARCHITECTURE

Q. How do you share reactive data across multiple unrelated components without prop drilling or third-party state managers?

ANSWER. In the past, sharing a user's login status across 15 different components required messy prop-drilling or bulky external libraries like Redux. Now, you just export `$state` variables directly from a plain JavaScript file. Any component can import that file and share the exact same live data instantly.

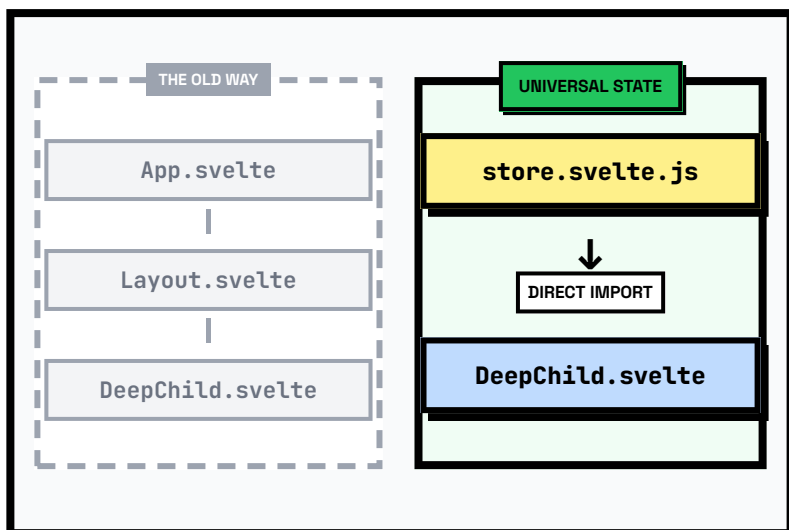
```
export const theme = $state({ dark: false });
```

store.svelte.js

```
<script>
  import { theme } from './store.svelte.js';
</script>

<button onclick={() => theme.dark = true}>
  Dark Mode
</button>
```

DeepChild.svelte



i Summary. Svelte 5 introduces **universal reactivity**.

This is useful because you can export `$state` directly from a plain JavaScript file. This lets any component across your entire app import and share live data instantly. It completely removes the need for messy prop drilling.

6 — Standard Events

EVENT HANDLING

Q. How do you attach an event listener to a DOM element in Svelte 5?

ANSWER. When you want to trigger a function when a user clicks a button, you used to need Svelte-specific syntax like `on:click`. Now, you use standard HTML event attributes like `onclick` and pass in a normal JavaScript function, making your code immediately readable to anyone who knows plain JavaScript.

Button.svelte

```
<script>
  function handleClick(e) {
    e.preventDefault();
    console.log("Clicked!");
  }
</script>

<button onclick={handleClick}>
  Click Me
</button>
```

THE OLD WAY

~~on:click~~

|preventDefault

SVELTE 5

onclick

Standard JS Event Object

- ❶ **Summary.** Svelte 5 drops the custom `on:` syntax in favor of plain old `onclick`. This simplifies things because you stop learning framework-specific magic. You just use standard, predictable JavaScript to handle events. This makes your code

portable and easy to read.

7 — Fragments: #snippet

REUSABILITY

Q. How do you pass reusable chunks of UI into a child component instead of using the old slot system?

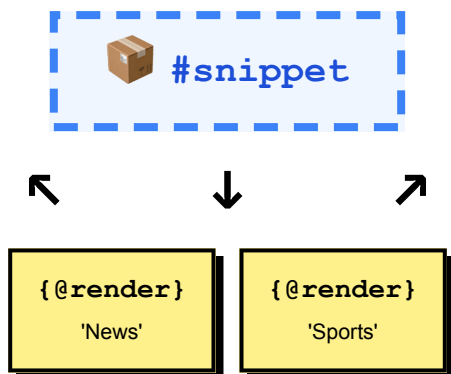
ANSWER. If you are building a complex `<Table>` component, you need the parent to dictate exactly how each row's data is formatted as HTML. Instead of relying on rigid, clunky `<slot>` tags, you define dynamic UI blocks using the `#snippet` syntax and pass them directly into the child component like a normal function.

```
{#snippet card(title, content)}
  <div class="ui-card">
    <h3>{title}</h3>
    <p>{content}</p>
  </div>
{/snippet}

{@render card("News", "Svelte 5 is out!")}
{@render card("Sports", "Local team wins!")}
```

App.svelte

BLUEPRINT FACTORY



i Summary. Snippets replace the old `<slot>` system. This is a game changer because they let you define mini, reusable chunks of UI right inside your component. This creates an incredibly

powerful and programmatic way to pass layout pieces around your app.

8 — Two-Way Flow: \$bindable

STATE SYNCHRONIZATION

Q. How do you explicitly allow a child component to modify a value passed down from its parent?

ANSWER. Normally, data only flows strictly down from parent to child. But if you have a custom `<TextInput>` component, you need the child to actively push the typed letters back up to the parent. You use the `$bindable` rune to explicitly create a two-way street that automatically keeps both synced.

```
<script>
  let { value = $bindable() } = $props();
</script>

<input bind:value={value} />
```

Input.svelte

```
<script>
  import Input from './Input.svelte';
  let name = $state('Alice');
</script>

<Input bind:value={name} />
```

App.svelte

Parent

name

Child

\$bindable()

- ❶ **Summary.** The `$bindable` rune creates a two-way street. This is important because it explicitly gives a child component permission to edit the parent's state. This keeps complex data perfectly synced without writing endless callback functions.

9 — Logging: \$inspect

DEVELOPER TOOLS

Q. How do you automatically log the old and new values of a state variable every time it changes for debugging?

ANSWER. When an object is changing unexpectedly and breaking your UI, manually writing `console.log()` inside every single update function is exhausting. You drop the `$inspect` rune into your component and pass it the broken variable. Svelte acts as a security camera, automatically logging the exact old and new values every time it shifts.

```
<script>
  let count = $state(0);
  let user = $state({ role: 'admin' });

  // Logs automatically on any change
  $inspect(count, user);
</script>
```

App.svelte



DEVELOPER CONSOLE

```
> $inspect initialization
count: 0  Object { role: "admin" }

> count updated
count: 1  Object { role: "admin" }
```

i Summary. Drop `$inspect` into any component to watch your variables. This is incredibly handy because it acts as a built-in debugger. It automatically prints out the exact old and new values to the console every single time your state changes.

10 — Teardown: Effect Cleanup

LIFECYCLE

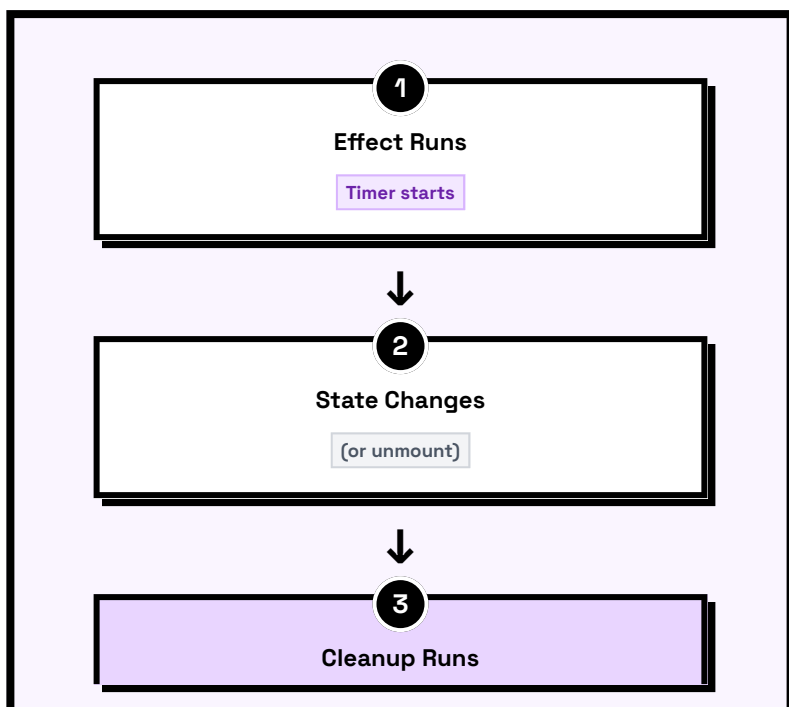
Q. How do you prevent memory leaks from timers or subscriptions created inside an `$effect`?

ANSWER. If you start a `setInterval` timer inside a component to build a stopwatch, that timer will keep firing forever—even if the user navigates to a totally different page, severely lagging the browser. You return a cleanup function from within the `$effect` block, guaranteeing the timer is destroyed the second the component leaves the screen.

Timer.svelte

```
<script>
  $effect(() => {
    const timer = setInterval(() =>
      console.log('tick'), 1000);

    // Return a function to clean up
    return () => clearInterval(timer);
  });
</script>
```



Timer stops

- ① **Summary.** Return a function from your **\$effect** to clean up tasks. This is important because it guarantees that existing timers or data subscriptions are safely destroyed when the component unmounts. This prevents massive memory leaks with zero extra boilerplate.

11 — Class Reactivity

STATE MANAGEMENT OOP

Q. How do you bundle reactive data together with the methods that modify it into a single, portable object?

ANSWER. If you are building a complex game character, having their health `$state` and their `takeDamage()` logic scattered across multiple files is an organizational nightmare. You declare class fields using the `$state` rune inside a standard JavaScript class, letting you bundle live data and complex logic into clean, reusable objects.

```
export class User {  
  name = $state('');  
  
  constructor(name) {  
    this.name = name;  
  }  
}
```

User.js

```
<script>  
  import { User } from './User.js';  
  const user = new User('Alice');  
</script>  
  
<input bind:value={user.name} />
```

Profile.svelte

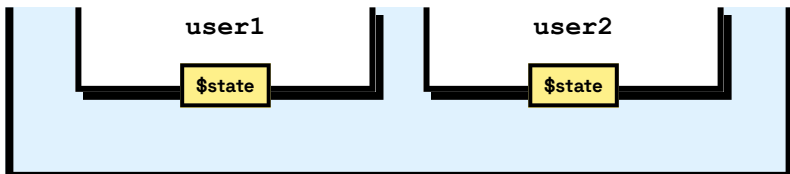
class User

INSTANCE



INSTANCE





- ① **Summary.** `$state` now works perfectly inside a JavaScript `class`. This is powerful because it lets you bundle your reactive data together with the custom logic that updates it. You can now create clean, reusable objects to pass around your app.

12 — Ignoring Changes: untrack

REACTIVITY PERFORMANCE

Q. How do you read a reactive variable inside an `$effect` without causing the effect to re-run when that variable changes?

ANSWER. By default, Svelte automatically tracks every reactive variable you touch inside an `$effect`. If you want to use a variable's value (like sending a `sessionToken` inside a `fetch()` call) *without* letting its changes trigger the effect again, you must hide it from Svelte. You do this by passing a function into `untrack()`. It safely returns the current value, but completely blocks Svelte from tracking the dependency.

Analytics.svelte

```
<script>
  import { untrack } from 'svelte';
  let actionCount = $state(0);
  let sessionToken = $state('xyz-987');

  $effect(() => {
    // 1. Read the token while Svelte is
    "blindfolded"
    // untrack() executes the function and returns
    its value, but blocks reactivity.
    const currentToken = untrack(() => sessionToken);

    // 2. Perform the action (this ONLY re-runs when
    actionCount changes)
    fetch('/api/log', {
      method: 'POST',
      body: JSON.stringify({ action: actionCount,
        token: currentToken })
    });
  });
</script>
```

actionCount



TRIGGERS RE-

```
untrack(() =>
  sessionToken)
```

Returns "xyz-987"

RUN

IGNORED
BY SVELTE

- ① **Summary.** `untrack()` lets you read a variable without Svelte watching it. This is essential because it gives you precise control. You can safely fetch data or log analytics using live variables without accidentally firing duplicate network requests every time those background variables update.

13 — Pre-rendering Effects

LIFECYCLE DOM

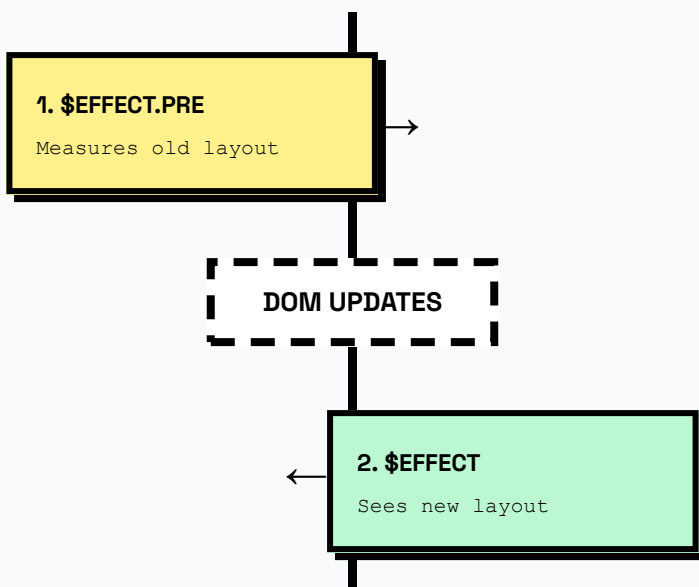
Q. How do you read layout dimensions or scroll positions immediately before the browser repaints the screen?

ANSWER. If you are building an auto-scrolling chat app, reading the scroll height during a normal `$effect` happens *after* the UI updates, causing a jarring visual flicker. You use the `$effect.pre` rune to measure layouts exactly a split-second *before* the browser repaints, allowing you to create butter-smooth UI adjustments.

Scroll.svelte

```
<script>
  let height = $state(0);

  $effect.pre(() => {
    // Runs BEFORE the DOM updates
    console.log("Old height:",
      document.body.scrollHeight);
  });
</script>
```



① **Summary.** `$effect.pre` fires right *before* the browser repaints the screen. This is critical because it lets you measure layouts or save scroll positions right before a UI shift happens. This allows you to create perfectly smooth animations and transitions.

14 — Dynamic Components

ARCHITECTURE ROUTING

Q. How do you render a component dynamically based on a variable without using the verbose `svelte:component` tag?

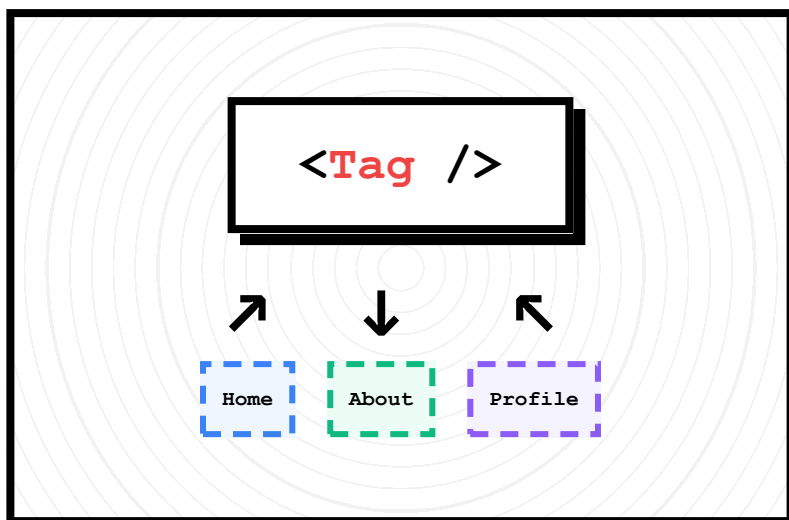
ANSWER. If you are building a dynamic dashboard where a user can choose to view a `<LineChart>` or a `<PieChart>`, you need to swap the UI out on the fly. Instead of writing massive `if/else` blocks, you assign the chosen component to a capitalized variable and render it directly as an HTML tag.

```
<script>
  import Home from './Home.svelte';
  import About from './About.svelte';

  let Tag = $state(Home);
</script>

<!-- Renders whichever component is currently
assigned -->
<Tag />
```

App.svelte



Summary. Svelte 5 removes the verbose

`<svelte:component>` tag entirely. This is fantastic because you can now assign an imported component straight to a **capitalized variable**. You can render it directly, making dynamic routing cleaner and more native-feeling.

15 — Error Boundaries

ARCHITECTURE SAFETY

Q. How do you catch fatal JavaScript errors thrown by a child component and display a fallback UI instead of crashing the app?

ANSWER. If you fetch data for a weather widget and the third-party API returns malformed garbage, the resulting JavaScript error will instantly white-screen your entire application. You wrap the widget inside a `<svelte:boundary>` tag, catching the localized error and displaying a safe fallback UI so the rest of the app stays alive.

App.svelte

```
<script>
  import RiskyWidget from './RiskyWidget.svelte';
</script>

<svelte:boundary>
  <RiskyWidget />

  <!-- Fallback UI if RiskyWidget throws an error -->
  {#snippet failed(error, reset)}
    <p>Crash: {error.message}</p>
    <button onclick={reset}>Retry</button>
  {/snippet}
</svelte:boundary>
```

SAFE APP ZONE



① **Summary.** Svelte 5 introduces the `**<svelte:boundary>**` wrapper. This is a lifesaver because it catches fatal JavaScript errors inside child components and displays a clean fallback UI instead. This guarantees that one bad widget never crashes your entire app again.

16 — Shallow State: \$state.raw

PERFORMANCE REACTIVITY

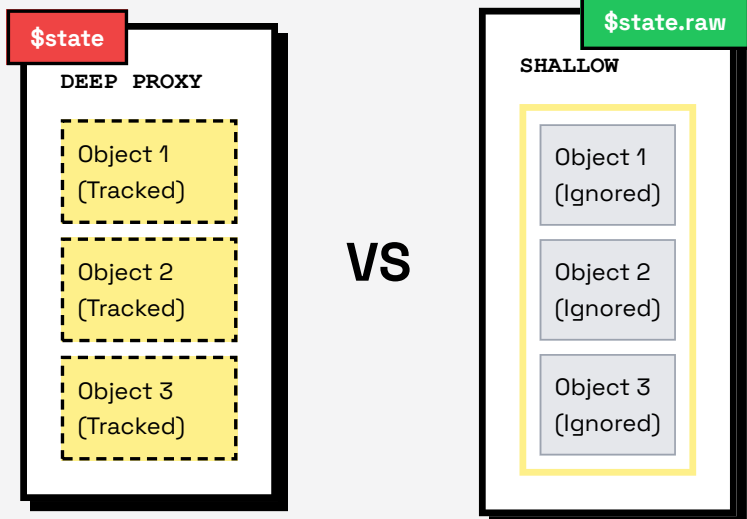
Q. How would you handle a massive array of 10,000 data points in Svelte 5 without causing severe memory and performance overhead?

ANSWER. Normally, `$state` builds a heavy 'proxy' around every single nested property of an object to track microscopic changes. But if you have 10,000 data points for a financial chart, that tracking overhead will freeze the browser. You use `$state.raw` to completely skip deep tracking, guaranteeing blazing fast renders as long as you swap the whole array out at once.

```
<script>
  // Perfect for thousands of data points
  let data = $state.raw([
    { x: 1, y: 10 },
    { x: 2, y: 15 }
  ]);

  function loadNewData(newData) {
    // You MUST replace the whole array.
    // data.push() will not trigger updates.
    data = newData;
  }
</script>
```

ChartData.svelte



- ① **Summary.** Deep reactivity is expensive on massive datasets. This matters because `$state.raw` lets you opt out of that

overhead. You get blazing fast renders for large lists as long as you swap the whole array instead of mutating individual items.

17 — Manual Reactivity: \$effect.root

ADVANCED LIFECYCLE

Q. How do you use reactivity like `$effect` outside of a component lifecycle (e.g., in a global store) without causing memory leaks?

ANSWER. Normally, effects live inside components so Svelte can automatically destroy them when the component unmounts. But if you create global state in a plain `.js` file and want an effect there (e.g., to auto-save to `localStorage`), Svelte blocks it to prevent memory leaks. You wrap it in `$effect.root`, which hands you a manual cleanup function to run when you're done.

```
import { $effect } from 'svelte';

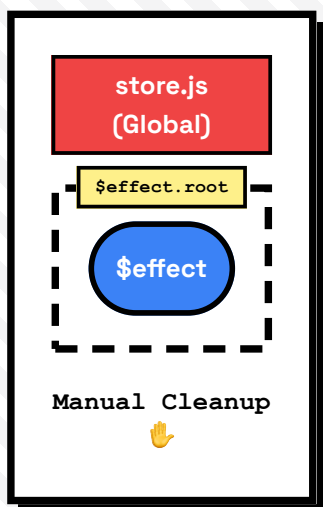
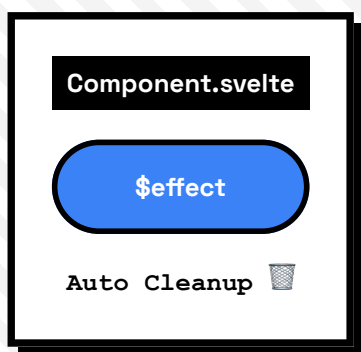
export function createGlobalLogger(stateValue) {
  // Returns a manual cleanup function
  const cleanup = $effect.root(() => {

    $effect(() => {
      console.log("State changed:", stateValue());
    });

  });

  return cleanup;
}
```

logger.js



① **Summary.** Normal effects die when their component unmounts. This is crucial because `$effect.root` lets you safely build global state managers or listeners in plain JavaScript files. It hands you the exact cleanup function you need to prevent memory leaks.

18 — Complex Formulas: \$derived.by

REACTIVITY LOGIC

Q. How do you calculate a derived value that requires multiple lines of logic, temporary variables, or loops?

ANSWER. Normally, a `$derived` value is just a single math equation. But if you need to filter a massive list of products based on five different search criteria, you need `if` statements and loops. You use `$derived.by` to run a massive, multi-line JavaScript function that unlocks the full power of the language while keeping the result perfectly cached.

Stats.svelte

```
<script>
  let numbers = $state([1, -5, 10, 42, -3]);

  // Use .by() when you need a full function block
  let positiveTotal = $derived.by(() => {
    let sum = 0;
    for (let num of numbers) {
      if (num > 0) sum += num;
    }
    return sum;
  });
</script>
```

\$derived

One line
only.
`x * 2`

\$derived.by

Full
function.
Loops
allowed.
If
statements.
Must
return.

① **Summary.** Sometimes a simple math expression isn't enough to calculate a derived value. This is powerful because `$derived.by` unlocks the full power of JavaScript inside your reactivity. You can write complex parsing or filtering algorithms that still auto-update efficiently.